

Discovery Hub — Engineering & Architecture

Implementation Plan

A technical memo for the Purdue summer research team (3 students + instructor), with answers for the investor on data, compute placement, build speed, and reproducibility.

TL;DR

- **Build the three-layer "investor-deck" vision incrementally on top of the existing retrieve-then-rerank package, which maps cleanly to Layer 2.** Layer 1 (knowledge graph) and Layer 3 (multi-agent RAG) are net-new, but every layer can be scaffolded with deterministic mocks against real data in days, with a credible end-to-end demo in ~10–12 weeks. The long poles are knowledge-graph construction/R-GCN and multi-agent orchestration, not embedding fine-tuning (already solved).
- **Use a hybrid compute pattern: heavy batch jobs (embedding millions of docs, R-GCN training, 4B LoRA fine-tune) on Anvil's A100/H100 nodes; persistent demo + always-on LLM/RAG serving on the Drew server.** The full set of training jobs is comfortably inside the ~6,000 GPU-hour allocation — embedding ~40M abstracts is tens of A100-hours and an R-GCN run on a ~2M-node graph is ~1–6 GPU-hours, so the budget is generous, not tight.
- **On reproducibility: be honest that byte-level determinism is not guaranteed even at temperature 0 (floating-point non-associativity + batch-variant GPU kernels), but semantic/functional stability is achievable and measurable.** Build a QA harness that reports exact-match rate, embedding cosine drift, retrieval Jaccard/rank-correlation, and LLM semantic-equivalence across N repeated runs. That measured stability — not a false claim of perfect determinism — is what gives the product credibility.

Key Findings

1. **The existing package is Layer 2.** The discovery_finetune work (schema normalization of AUTM/ClinicalTrials.gov v2/USPTO, Gemini synthetic-query generation, hard-negative mining, MNRL fine-tuning of Qwen3-Embedding-0.6B with a 4B LoRA path, Recall@k/MRR@10 eval) is precisely the "hybrid retrieval + neural reranking" middle layer. The team should formally wrap it as the retrieval/ranking service and build Layers 1 and 3 around it.

2. **Data is freely and legally available at both MVP and full scale.** Per the OpenAlex Developers docs ("Snapshot data format"): "The gzip-compressed snapshot is approximately 330 GB and decompresses to about 1.6 TB," with the free public snapshot refreshed quarterly (CC0). USPTO/PatentsView is CC-BY 4.0 (grants ~100 GB, pre-grant ~26 GB, ~500 GB unzipped with full text). ClinicalTrials.gov v2 is a free JSON API: per its "Trends and Charts on Registered Studies" (Dec 2025), "ClinicalTrials.gov currently lists 590,801 studies with locations in all 50 states and in 226 countries and territories." NIH RePORTER ExPORTER is free bulk; per SBIR.gov "Data Resources," "All Award data can be downloaded as Award data with abstract information (290 MB) and Award data without abstract information (65 MB)." Per the Allen Institute for AI's S2AG paper (Wade et al., 2022), "S2AG currently contains over 205M publications, 121M authors, and nearly 2.5B citation edges." The 250M-work OpenAlex corpus filters to a tractable ~43M biomedical subset (medicine + biology) via Topics/Concepts.
3. **Anvil is an A100/H100 batch HPC, not a serving platform.** Per RCAC "Compute: Anvil," the system has "16 nodes each with four NVIDIA A100 Tensor Core GPUs providing 1.5 PF of single-precision performance," running on 1,000 dual 64-core AMD Epyc "Milan" nodes (5.3 PF peak, plus 32 large-memory 1 TB-RAM nodes). Each base A100 is 40 GB. A separate "Anvil AI" expansion adds 21 nodes × 4× H100 80GB SXM (request "Purdue Anvil AI" for H100s). Anvil is batch-scheduled (Slurm) and explicitly not designed for always-on interactive web services — though the Anvil Composable Subsystem (Kubernetes/Rancher, with an A100 80GB node) can host persistent services. The Drew server (2× 12 GB GPUs) is the right place for the always-on demo and self-hosted LLM, but 12 GB caps you to ~7–8B models at 4-bit.
4. **The deck's headline targets are ambitious for an SBIR-stage student team.** +20% Precision@10/NDCG, ≥95% citation accuracy, and <5% hallucination are achievable as *aspirations* but should be reframed as measured interim targets. Public RAG faithfulness benchmarks (RAGBench, FaithBench, RAGTruth) show hallucination detection itself is hard and component datasets contain 1–20% hallucinations; [arxiv](#) the honest play is to *instrument and report* these numbers, not assert them.
5. **Nondeterminism is real but bounded and explainable.** The 2025 Thinking Machines "Defeating Nondeterminism in LLM Inference" work pinpoints batch-variant GPU kernels as the dominant cause of run-to-run variation even at temperature 0, and ships batch-invariant kernels that make vLLM bit-reproducible. This is the centerpiece of an honest, sophisticated investor answer.

Details

QUESTION 1 — DATA: What to download, in what quantities, for each layer

Layer 1 – Evidence Integration (knowledge graph) data sources

Source	Access mechanism	License	MVP volume
OpenAlex (works, authors, institutions, sources, funders, topics)	AWS S3 anonymous: <code>aws s3 sync 's3://openalex' --no-sign-request (us-east-1)</code> ; quarterly free snapshot	CC0 (public domain)	Biomedical subset only: filter Works by Topic/Concept (Medicine, Biology) ≈ a few hundred GB pre-filter, ~tens of GB c filtered JSONL
PatentsView / USPTO Open Data Portal (migrating to data.uspto.gov on 2026-03-20)	Bulk TSV/zip downloads; PatentSearch API	CC-BY 4.0	Pharma-relevant CPC subset (e.g., A61K/A61P/C07/C12N) – single-digit GB
ClinicalTrials.gov API v2	REST JSON, cursor pagination, <code>pageSize=1000</code> , no key	Public domain	All pharma/interventional trials (subset of 590,801)
NIH RePORTER / ExPORTER	Bulk zip per fiscal year + API v2	Public domain	Recent FYs of project/abstract/link/pate files
SBIR.gov awards	Bulk file (290 MB with abstracts / 65 MB without) + Award/Company/Topic APIs	Public domain	Full file – it's tiny
Semantic Scholar S2AG	Datasets API (gzipped JSON), monthly snapshots	ODC-BY (per-record varies)	abstracts + papers + citations datasets for biomedical IDs
AUTM Innovation Marketplace (AIM/Flintbox)	Web portal (31,000+ technologies posted); no clean bulk API	Proprietary/portal terms	Scrape/normalize the pharma-relevant subset already in your schema
Crunchbase (investor/company signals)	Paid API/licensed CSV; no free bulk	Commercial license required	Defer – use free signals (SBIR, RePORTER, patent assignees) for MVP

Scoping the 250M-work OpenAlex corpus to a tractable pharma/biomedical subset.

Three complementary filters:

- **By Topic/Concept:** OpenAlex has ~4,500 Topics [Openalex](#) (and the deprecated ~65k Concepts). [Openalex](#) Filtering Works to the "Medicine" and "Biology" branches yields roughly **43M papers** (the published literature reports ~24.3M in medicine and ~18.4M in biology from a recent OpenAlex snapshot), with chemistry adding ~4.2M. A tighter therapeutic-area cut (e.g., subfield Epidemiology id 2713, [Openalex](#) or pharmacology/oncology topics) drops this to single-digit millions.
- **By linkage to patents/trials:** keep only works cited by your pharma patent set, linked from ClinicalTrials.gov references, or acknowledged in NIH RePORTER grant-publication links. This produces a high-precision "translational" core of perhaps 1–5M works.
- **By MeSH/therapeutic area:** join on PubMed IDs (present in OpenAlex `ids`) and filter by MeSH descriptors for the target indications.

Recommended MVP vs. full-scale data plan:

- **SBIR-stage demo/MVP (fits on Drew, ~1–2 TB disk):** Pharma-CPC patent subset (single-digit GB), all ClinicalTrials.gov + SBIR + recent RePORTER (a few GB), and a **1–5M-work biomedical OpenAlex slice** (tens of GB). Total raw data: well under ~200 GB; comfortably local.
- **Full-scale production (lives on Anvil scratch/Ceph or cloud):** Full OpenAlex (~330 GB gzip / 1.6 TB), full USPTO full-text (~500 GB unzipped), S2AG (hundreds of GB). Total **~2–3 TB** working set. Use Anvil's Ceph/Lustre scratch and DuckDB/Parquet conversion to query without full decompression. Note Anvil scratch is **not backed up** — keep raw snapshots on Project space or re-pull from S3.

Layer 2 — Retrieval & Ranking data

This layer consumes the normalized document schema you already produce, plus the Gemini-generated synthetic queries and mined hard negatives. The only new data need is **scaling the corpus to the biomedical slice above** and generating embeddings for it (see Q2 for placement).

Layer 3 — Explanation & Workflow data

Needs no new bulk corpus — it operates over retrieved documents in strict RAG mode. What it *does* need is a small, curated **evaluation/QA set** (50–500 query → expected-evidence pairs) for measuring citation accuracy, faithfulness, and stability. Build this by hand from a few therapeutic areas; it is the single highest-leverage data artifact for the investor demo.

QUESTION 2 — COMPUTE PLACEMENT: Anvil vs. Drew, stage by stage

Hardware ground truth.

- **Anvil GPU:** 16 nodes × 4× NVIDIA A100 (40 GB each); charge unit is the SU where **1 GPU-hour ≤ 120 GB-memory = 1 SU**. GPU nodes are shared. ~6,000 GPU-hours is what ~400,000 ACCESS Explore credits convert to on Anvil.
- **Anvil AI:** 21 nodes × 4× NVIDIA H100 80GB SXM (request "Purdue Anvil AI"; ~9× faster than A100 for AI workloads per RCAC).
- **Anvil Composable Subsystem:** Kubernetes/Rancher private cloud with an A100 80GB node — designed for persistent services/gateways/inference, the one place on Anvil suited to always-on hosting.
- **Drew:** 2× 12 GB GPUs — always-on, but 12 GB caps you to ~7–8B LLMs at 4-bit (e.g., Llama-3.1-8B Q4/Q8, Qwen3-8B, Mistral) plus the 0.6B embedding model.

Pipeline stage	Where	Why
(a) Bulk ingestion, parsing, KG construction	Anvil CPU nodes (128-core Milan, 256 GB RAM; 1 TB large-mem nodes available) + Ceph/scratch	I/O- and RAM-heavy, not GPU-heavy. Use Anvil's CPU partition + DuckDB/Parquet. Drew lacks the disk and RAM for 1.6 TB+ working sets.
(b) Embedding generation at scale	Anvil A100/H100 (batch)	Embedding ~40M abstracts with Qwen3-0.6B is ~tens of A100 GPU-hours (see budget below). Batch job, perfect HPC fit. Persist vectors to disk; ship the index to Drew.
(c) R-GCN / GNN training	Anvil A100 (single GPU with neighbor sampling)	Full-batch on millions of nodes needs >40 GB and is intractable on small GPUs; with neighbor sampling/GraphSAINT/Cluster-GCN, ≤16 GB is sufficient and even ~4–5 GB works for very large graphs. Runs fit on one A100; could even run on Drew's 12 GB with sampling, but Anvil is faster for sweeps.
(d) Contrastive fine-tuning of embedding models	0.6B → Drew (fits 12 GB); 4B LoRA → Anvil A100/H100	Qwen3-Embedding-0.6B trains comfortably in 12 GB. The 4B LoRA model wants ≥40 GB headroom for batch sizes that make MNRL effective → Anvil.
(e) Multi-agent LLM inference / RAG serving	Drew (always-on) via vLLM/TGI, optionally Anvil Composable for heavier batch eval	Anvil's batch scheduler is wrong for an interactive demo. Self-host a 7–8B model at 4-bit on Drew with vLLM; call Gemini/OpenAI APIs for harder generations during development.

Recommended hybrid pattern: *Anvil* = the factory (parse, embed, train, batch-evaluate). *Drew* = the storefront (vector index, retrieval API, vLLM-served LLM, multi-agent orchestrator, web demo). Artifacts (embeddings, trained R-GCN weights, fine-tuned encoder) are produced on Anvil and rsync'd/Globus'd to Drew.

GPU-hour budget sanity check against ~6,000 hours (all numbers A100-equiv):

- **Embedding 40–50M biomedical abstracts** (Qwen3-0.6B, BF16): extrapolating from a measured ~70 docs/sec on an A10G (Luxical technical report, arXiv 2512.09015), ~178 A10G-hours → **~30–60 A100-hours**. Even at a conservative 200 docs/sec on A100, well under 100 hours. (Production confirmation that this is routine on single GPUs: the Eventual/Daft team embedded "millions of text documents" with Qwen3-Embedding-0.6B at near-100% utilization on 1 A10G per worker.)
- **R-GCN training run** on a ~2M-node/21M-edge heterogeneous graph (ogbn-mag scale: 1,939,743 nodes / 21,111,007 edges) with neighbor sampling: **~1–6 GPU-hours per run**. A comparable hetero-GNN (LDHGNN) "took an average of 5 hours and 40 minutes per run" on a slower Tesla P100 (IMPACT GNN, arXiv 2411.10957); A100 is faster, and R-GCN on ogbn-mag converges in single-digit-to-low-tens of epochs. Budget for 50–100 runs across sweeps → a few hundred hours.
- **Memory note for R-GCN:** the OGB paper states full-batch GNNs "require more than 40GB of GPU memory to train, which is intractable on most of the GPUs available today," but their official mini-batch benchmarks ran on an **11 GB GPU**, and SHADOW-GNN reports "even for ogbn-papers100M, 4GB of GPU memory and 128GB of RAM are sufficient." **A 12 GB Drew GPU therefore suffices for R-GCN with sampling**; full-batch is the only regime needing A100-class memory.
- **4B embedding LoRA fine-tune:** order of tens of GPU-hours per run; budget a few hundred for experiments.
- **Domain-adaptation of an LLM (if attempted — optional):** LoRA/QLoRA on a 7–8B model is tens to low-hundreds of GPU-hours; full fine-tune is out of scope.
- **Total realistic consumption: ~1,000–2,000 GPU-hours** including generous re-runs and hyperparameter search, leaving comfortable headroom in the 6,000-hour allocation. **The allocation is generous, not the binding constraint** — disk, data engineering, and student time are the real limits.

QUESTION 3 — BUILD & TEST SPEED: A realistic phased timeline

The team favors robust, end-to-end-tested infrastructure validated with deterministic mocks against real data before scaling — the right instinct. Map each layer to existing OSS so students integrate rather than invent.

Accelerating frameworks per layer:

- **Layer 1 (KG):** DuckDB/Parquet for ingestion; a graph store (Neo4j, or just PyG/DGL `HeteroData` tensors); entity resolution leans on PatentsView's pre-disambiguated

inventor/assignee data and OpenAlex's disambiguated authors/institutions — *do not rebuild disambiguation*.

- **Layer 2 (retrieval/ranking)**: sentence-transformers (already in use); PyTorch Geometric or DGL for R-GCN (PyG's `RGCNConv` / `FastRGCNConv` with basis decomposition; DGL has a built-in efficient R-GCN module); FAISS/Qdrant for the vector index.
- **Layer 3 (multi-agent RAG)**: LangGraph for stateful multi-agent orchestration (retrieval / expertise-gap / reranking / policy-safety agents); LlamaIndex for ingestion+retrieval [Internal Technologies](#) and built-in citation/faithfulness evaluators; vLLM or TGI for LLM serving; RAGAS for faithfulness/citation metrics. The pragmatic 2026 pattern reported across multiple framework comparisons is **LlamaIndex for retrieval + LangGraph for orchestration + RAGAS/LangSmith for eval**.

Phased roadmap (3 students + instructor):

Phase	Duration	Deliverable	What's prototyped vs. hardene
0. Foundation	Days 1–5	Repo, CI, deterministic mock interfaces for all 3 layers; data-download scripts (S3 sync, ClinicalTrials.gov, RePORTER, SBIR)	Mocks + contracts in days
1. Layer 2 productionization	Weeks 1–3	Existing retrieve-rerank wrapped as a service over the biomedical slice; FAISS index; Recall@k/MRR@10 dashboard	Mostly done hardening + scale-up
2. Layer 1 KG (the long pole #1)	Weeks 2–7	Heterogeneous graph (technologies/inventors/orgs/experts/facilities) from PatentsView + OpenAlex + trials; R-GCN node/edge embeddings trained on Anvil	KG schema in ~1 week; entity linking + R-GCN training is the multi-week effort
3. Layer 3 multi-agent RAG (long pole #2)	Weeks 4–9	LangGraph pipeline with strict RAG, per-claim citations, provenance, confidence; vLLM-served 7–8B model on Drew	Single-agen RAG in days multi-agent + policy/safet + eval is multi-week
4. Stability/QA harness	Weeks 6–10	The reproducibility harness (Q4) wired into CI; investor-facing stability report	~2 weeks once layers

Phase	Duration	Deliverable	What's prototyped vs. hardene
			exist
5. Integrated demo + hardening	Weeks 10–12	End-to-end Drew demo; Anvil batch jobs for refresh; eval on curated QA set	Continuous

Long poles: (1) KG construction + R-GCN (entity resolution, schema design, getting R-GCN to train stably and *improve* retrieval), and (2) multi-agent orchestration with reliable citations. Embedding fine-tuning is **not** a long pole — it's already working. A credible end-to-end demo in **~10–12 weeks** is realistic; production hardening of all three layers is a multi-quarter effort.

Honest target-setting for the investor: Treat the deck's +20% Precision@10/NDCG, ≥95% citation accuracy, <5% hallucination as **measured goals with interim milestones**, not guarantees. Realistic interim targets: demonstrate a *statistically meaningful* lift over a BM25/keyword baseline (even +5–10% P@10 is a strong early result), citation-attribution accuracy measured on the curated QA set with a stated confidence interval, and a faithfulness score from RAGAS/HHEM-style evaluation with the hallucination rate reported honestly (public RAG datasets such as RAGBench run 1–20% hallucination across component sets).

QUESTION 4 – STABILITY / DETERMINISM / REPRODUCIBILITY

PART A – Why LLMs and embedding models are (and aren't) deterministic

(i) **Temperature and sampling.** With stochastic sampling (temperature > 0, top-p/top-k), outputs vary by construction because tokens are drawn from a probability distribution. **Greedy decoding (temperature = 0 / argmax)** removes that deliberate randomness — but it does **not** guarantee identical outputs run-to-run, because the underlying floating-point logits can shift slightly and flip the argmax.

(ii) **Sources of nondeterminism even at temperature 0.** The dominant, well-documented causes:

- **Floating-point non-associativity:** $(a+b)+c \neq a+(b+c)$ in floating point. GPU reductions (matmul, RMSNorm, attention) sum in parallel, and the *order* of summation can change between runs.
- **Batch-size-dependent numerics ("batch invariance"):** The 2025 Thinking Machines analysis ("Defeating Nondeterminism in LLM Inference," Horace He et al.) identifies the real culprit: standard kernels are **not batch-invariant**, so a request's output depends on how many *other* requests were batched with it — which varies with

server load. Verbatim: "the primary reason nearly all LLM inference endpoints are nondeterministic is that the load (and thus batch-size) nondeterministically varies." Tiny FP differences then "flip argmax in greedy decoding → completely different outputs." [NextBigFuture](#) Their fix — batch-invariant kernels (now adopted in vLLM/SGLang) — gives bitwise-reproducible inference but at a steep throughput cost: an independent LMSYS/SGLang quantification (arXiv 2601.17768) found batch-invariant kernels cause throughput to "collapse by 56% to about 415 tokens/s."

- **Kernel/library/version differences:** cuBLAS, cuDNN, and attention implementations (FlashAttention) can change reduction order across versions or hardware; the Triton FlashAttention backward even differs algorithmically from the paper. Different GPU architectures (x86 vs ARM, A100 vs H100) produce different rounding.

(iii) **Embedding model determinism.** Embeddings are generally **more reproducible** than generative outputs because there is no token-by-token error accumulation — but they are **not bit-identical across hardware**. A reproducibility experiment (Valori, arXiv 2512.22280) ran identical sentence-transformers code on x86 vs ARM64 and observed measurably different vectors due to floating-point interference in the forward pass. On the *same* hardware + library + batch size, embeddings are typically stable to within tiny cosine drift; the practical risk is **drift** when you change GPU, library version, batch size, or precision (FP16/BF16/INT8). The product-relevant fact: embeddings can drift enough to reorder near-tied retrieval results, even when they're "the same" to 4–5 decimal places.

(iv) **API-served closed models vs. self-hosted.** With **API models (Gemini, OpenAI)** you control neither batching, nor hardware, nor kernel versions — so even at temperature 0 you cannot guarantee reproducibility, and the provider can silently change the model. With **self-hosted models (vLLM/TGI on Drew)** you control hardware, library versions, batch size, and can adopt batch-invariant kernels — so you can get *much* closer to reproducibility. **This is a strong argument for self-hosting the explanation LLM on Drew for any claim that depends on stability.**

(v) **Seed-setting and its limits.** `torch.manual_seed`, `numpy.random.seed`, `torch.cuda.manual_seed_all`, `torch.use_deterministic_algorithms(True)`, `torch.backends.cudnn.deterministic=True`, `cudnn.benchmark=False`, disabling TF32, and `CUBLAS_WORKSPACE_CONFIG=:4096:8` [PyTorch](#) (required for deterministic cuBLAS on CUDA ≥10.2 — PyTorch raises a RuntimeError if it's unset) together remove *seed-based and algorithm-selection* nondeterminism. **Limits:** they don't fix cross-hardware or cross-version differences, some ops have no deterministic implementation [Dronelab](#) (they raise errors), results aren't reproducible between CPU and GPU, and deterministic kernels are **slower** (documented severe slowdowns in some torch versions). Seeds make *training* reproducible on a fixed stack; they do **not** by themselves make *served inference* batch-invariant.

PART B — A concrete, demonstrable stability/QA harness

Build a harness that runs the same N queries K times (e.g., N=100 curated queries, K=20 repeats) under controlled conditions and reports stability per stage. The key framing for the investor: **distinguish byte-level/exact determinism from semantic/functional stability, and measure both.**

Stage 1 — Embedding determinism.

- *Test*: Encode the same texts K times; also across batch sizes and (if available) both Drew GPUs.
- *Metrics*: exact-match rate of vectors; **max/mean cosine drift** ($1 - \cos$) across runs; count of cases where drift changes top-k retrieval order.
- *Target*: cosine drift $< 1e-6$ on fixed hardware/batch; report the (larger) cross-hardware drift honestly.

Stage 2 — Retrieval/ranking determinism.

- *Test*: Run retrieval K times on identical embeddings + index.
- *Metrics*: **Jaccard overlap** of the top-k result sets across runs; **Kendall's τ** / **Spearman rank correlation** of rankings; exact-match rate of the ordered top-10.
- *Target*: Jaccard = 1.0 and $\tau = 1.0$ on a fixed index (FAISS exact search is deterministic; approximate/ANN indexes may not be — disclose which you use).

Stage 3 — LLM explanation determinism.

- *Test*: Generate K times at temperature 0 for each query, on the self-hosted Drew model, with seeds and deterministic flags set; optionally compare with/without batch-invariant kernels.
- *Metrics*: **exact-match (byte-identical) rate**; **semantic-equivalence rate** (embedding cosine of outputs $\geq$ threshold, or an LLM-judge agreement); **citation-set stability** (Jaccard of cited source IDs across runs — this is the metric that matters most for this product); faithfulness/hallucination via RAGAS or an HHEM-style faithfulness model.
- *Target*: citation-set Jaccard ≈ 1.0 (the *evidence* should be stable even if prose varies); high semantic-equivalence; byte-identical only as a stretch goal with batch-invariant kernels.

Maximum-reproducibility configuration (and its costs):

- Fixed seeds across python/numpy/torch/cuda;
`torch.use_deterministic_algorithms(True);`
`CUBLAS_WORKSPACE_CONFIG=:4096:8; cudnn.deterministic=True,`
`benchmark=False`; disable TF32; [Ingoniyama](#) **temperature=0**; **fixed batch size** (or batch-invariant kernels in vLLM); **pinned library/driver versions** (containerize); **fixed hardware** (pin to one Drew GPU); **cache** embeddings and LLM responses keyed by input hash.

- Costs: deterministic kernels and fixed/size-1 batching reduce throughput (batch-invariant vLLM measured ~56% throughput collapse; deterministic torch ops measurably slower); caching adds storage but is essentially free latency-wise and is itself a strong stability mechanism.

How to present stability honestly to the investor. The credible message is: *"For this product, what matters is that the **evidence and recommendations are stable**, not that the prose is byte-identical. We measure and report (a) embedding drift, (b) retrieval-set and ranking stability, and (c) citation-set stability and faithfulness across repeated runs. On a fixed, self-hosted stack we achieve near-perfect stability of the retrieved evidence and citations; the natural-language explanation may vary in wording while pointing to the same sources, and we can make it byte-reproducible with batch-invariant kernels at a throughput cost if a customer requires it."* This reframes "do you get the same answer twice?" from a yes/no trap into a measured, defensible engineering property — and the caching layer means *identical inputs return identical cached outputs* in production, which is often the real question being asked.

Recommendations

Stage 1 (Weeks 0–3) — Lock the foundation.

1. Wrap discovery_finetune as the Layer-2 service; stand up FAISS + a retrieval API on Drew.
2. Write data-download scripts and pull the **MVP biomedical slice** (1–5M OpenAlex works + pharma-CPC patents + all trials/SBIR/recent RePORTER) — keep it under ~200 GB so it lives on Drew.
3. Stand up the deterministic mock harness and CI now, before scaling.
Threshold to proceed: Layer-2 reproduces its Recall@k/MRR@10 numbers on the real slice.

Stage 2 (Weeks 2–7) — Build the KG and R-GCN on Anvil.

4. Construct the heterogeneous graph using pre-disambiguated PatentsView/OpenAlex entities; train R-GCN with neighbor sampling on a single A100 (budget ≤300 GPU-hours including sweeps).
5. Run the full-corpus embedding job on Anvil (≤100 A100-hours); ship vectors + R-GCN weights to Drew.
Threshold to proceed: R-GCN-augmented retrieval beats the text-only baseline on the curated eval set (even +5% P@10).

Stage 3 (Weeks 4–10) — Multi-agent RAG + stability.

6. Build the LangGraph pipeline (retrieval/expertise-gap/reranking/policy-safety agents), strict RAG, per-claim citations + provenance + confidence; serve a 7–8B model on Drew via vLLM.
7. Wire the stability harness (Q4) and RAGAS faithfulness into CI; produce the investor

stability report.

Threshold to proceed: citation-set Jaccard ≈ 1.0 across repeats and faithfulness above your stated interim bar.

Stage 4 (Weeks 10–12) — Integrate, demo, harden.

8. End-to-end Drew demo; scheduled Anvil batch jobs for corpus refresh; document everything.

Benchmarks that would change the plan:

- If R-GCN does **not** beat the text-only baseline after reasonable tuning → keep Layer 2 as the product, treat the KG as a metadata/filtering layer, and don't over-invest in GNNs.
 - If 12 GB on Drew proves too tight for an acceptable explanation model → use the Anvil Composable Subsystem (A100 80GB) for a hosted inference endpoint, or call an API for generation while keeping retrieval local.
 - If data volume blows past Drew's disk → move the index to Anvil Ceph and expose it via the Composable subsystem.
-

Caveats

- **Deck claims vs. SBIR-stage reality.** "250M+ works indexed," "1M+ patents + 5M+ publications," +20% P@10/NDCG, $\geq 95\%$ citation accuracy, $< 5\%$ hallucination are credible *vision* numbers but not guaranteed deliverables for a 3-student summer team. Index the $\sim 43\text{M}$ biomedical OpenAlex subset (not 250M) for the MVP, and present quality metrics as *measured with confidence intervals* against named baselines.
- **Anvil is not a web host.** Its batch scheduler and non-backed-up scratch make it wrong for persistent serving; only the Composable Subsystem is suited to always-on services. Plan the demo on Drew.
- **A100 = 40 GB on the base Anvil GPU nodes;** the 80 GB cards are on the newer H100 "Anvil AI" nodes and the Composable A100 — request the right resource for memory-hungry jobs.
- **Reproducibility numbers are hardware/stack-specific.** Any stability figure you quote is valid only for the pinned hardware + library versions you measured on; say so.
- **Embedding/throughput and R-GCN GPU-hour figures are extrapolations** from comparable public benchmarks (A10G/P100/T4), not measurements on your exact corpus and Anvil A100s — validate with a small pilot before committing the full budget.
- **Licensing diligence:** OpenAlex (CC0) and USPTO/PatentsView (CC-BY 4.0) are clean; AUTM/AIM and Crunchbase carry portal/commercial terms — confirm before

redistributing derived data in a commercial product.